

NOMBRE:

KATYRIS CARABALLO SARMIENTO

PRATICA:

Introducción a Postman (GET y POST)

PROFESOR:

FELIZ MANUEL LORA MARTINEZ

CURSO

QA MANUAL

FECHA DE ENTREGA

06/04/2026

Introducción a Postman (GET y POST)

 API a utilizar

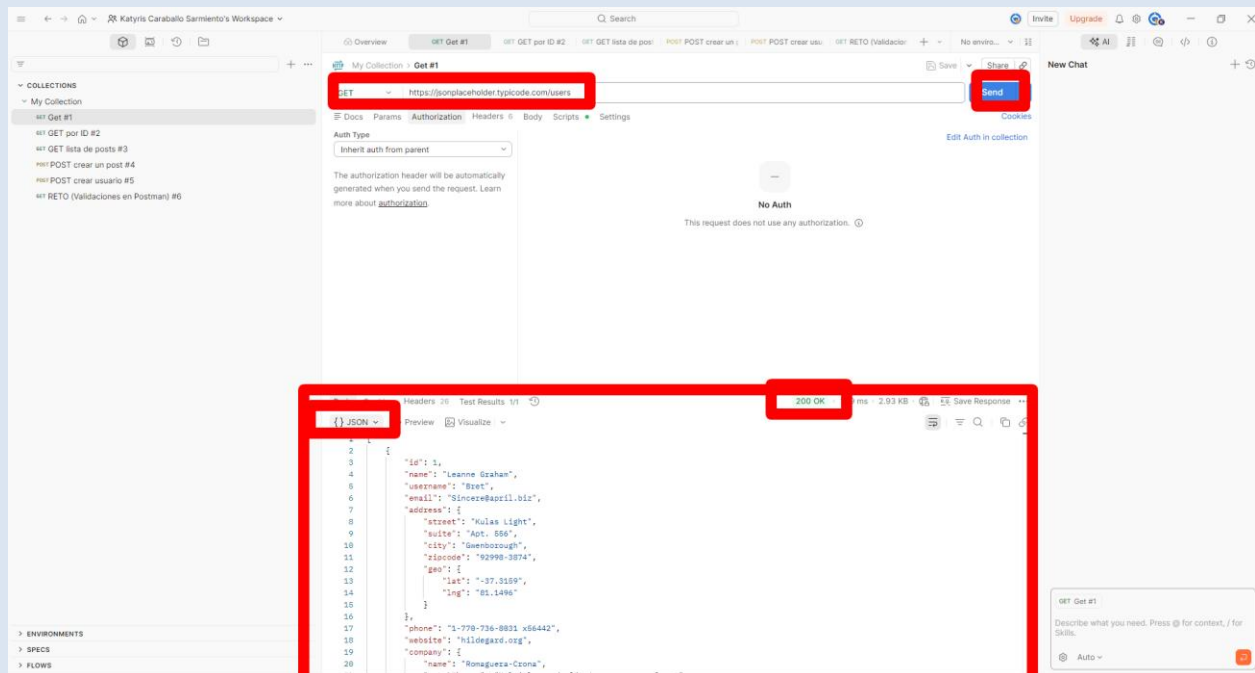
Usaremos esta API pública:

- https://jsonplaceholder.typicode.com

❖ EJERCICIO 1 (GUIADO) – GET todos los usuarios

Instrucciones

1. Abrir Postman
2. Seleccionar método ****GET****
3. Colocar la URL: <https://jsonplaceholder.typicode.com/users>
4. Presionar ****Send****
5. Resultado esperado



* **Código:** **200 OK**

* **Respuesta:** Lista de usuarios en formato JSON

❖ EJERCICIO 2 – GET por ID

Instrucciones

Realiza una petición para obtener el usuario con ID 1

- **Endpoint:** <https://jsonplaceholder.typicode.com/users/1>

-Validaciones

* **Código:** **200 OK**

* Debe mostrar un solo usuario

* El ID debe ser 1

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** <https://jsonplaceholder.typicode.com/users/1>
- Status Code:** 200 OK
- Response Body (JSON):**

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  },
  "phone": "1-770-736-8031 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net"
  }
}
```

❖ EJERCICIO 3 – GET lista de posts

Instrucciones

Consulta todos los posts

Endpoint: <https://jsonplaceholder.typicode.com/posts>

Validaciones

- * **Código:** **200 OK**
- * Debe devolver múltiples registros
- * Cada registro debe tener: `userId`, `id`, `title`, `body`

The screenshot shows the Postman interface with a GET request to `https://jsonplaceholder.typicode.com/posts`. The response status is **200 OK**. The response body is a JSON array of three post objects, each containing `userId`, `id`, `title`, and `body` fields.

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
},
{
  "userId": 1,
  "id": 2,
  "title": "qui est esse",
  "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
},
{
  "userId": 1,
  "id": 3,
  "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
  "body": "et lusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis"
}
```

❖ EJERCICIO 4 – POST crear un post

Instrucciones

1. Cambiar el método a ****POST****
2. URL: <https://jsonplaceholder.typicode.com/posts>
3. Ir a ****Body → raw → JSON****
4. Enviar este JSON:

```
`json
```

```
{
```

```
"title": "Mi primer post",
```

```
"body": "Aprendiendo Postman",
```

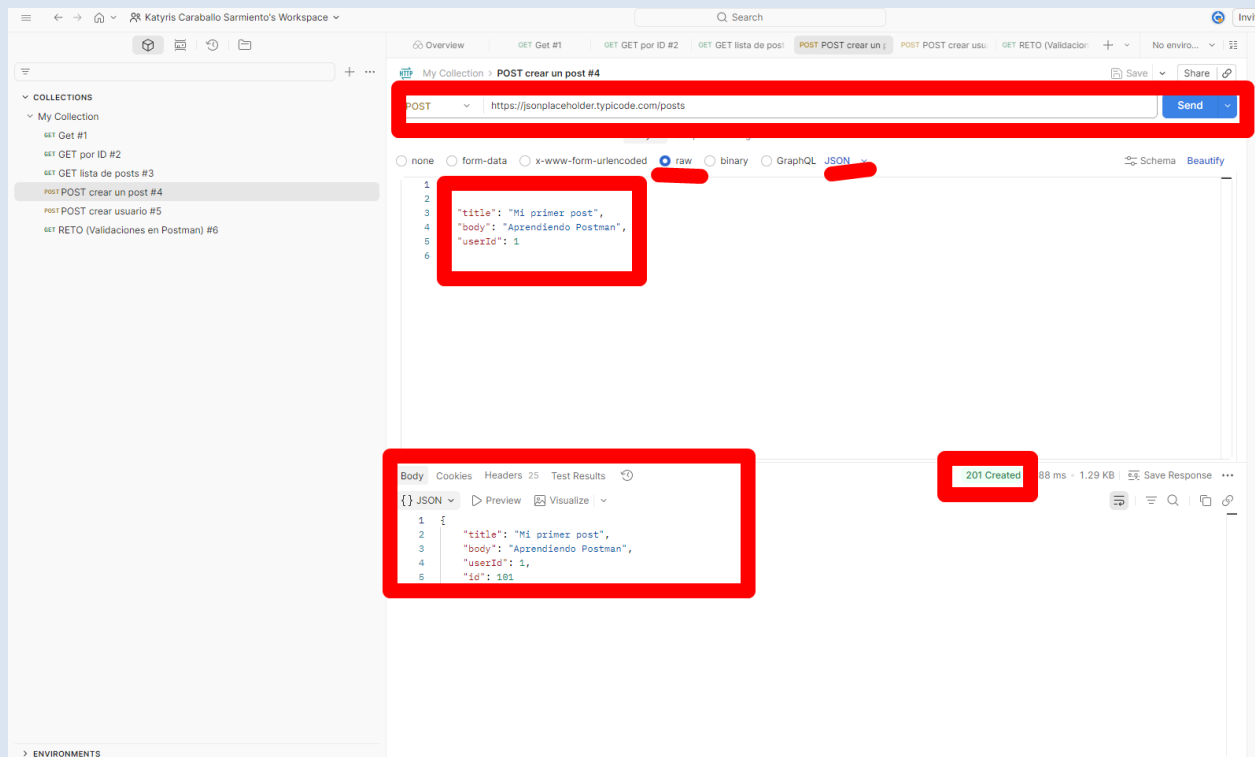
```
"userId": 1
```

```
}
```

```
...
```

Validaciones:

- * Código: ****201 Created****
- * Debe devolver el **objeto creado**
- * Debe incluir un **`id`**



❖ EJERCICIO 5 – POST crear usuario

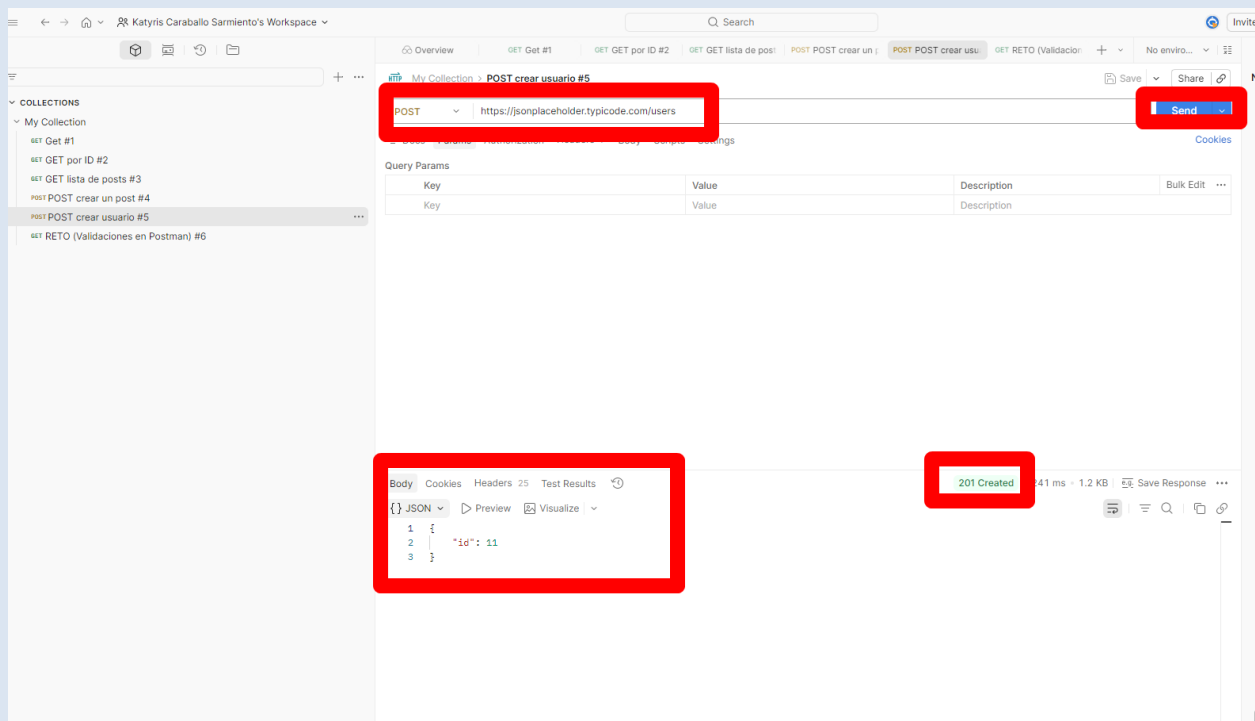
Instrucciones

Crear un nuevo usuario con los datos que desees

Endpoint: <https://jsonplaceholder.typicode.com/users>

Validaciones:

- Código: **201**
- Debe devolver los datos enviados
- Debe incluir un `id`



❖ EJERCICIO 6 – RETO (Validaciones en Postman)

Instrucciones

En cualquier request (GET o POST), agregar validaciones en la pestaña ****Tests****

Ejemplo:

```
```\javascript
pm.test("Status code es 200", function () {
 pm.response.to.have.status(200);
});
```
```

Objetivo

- Validar status code automáticamente
- Confirmar que la respuesta no esté vacía

RETO (Validaciones en Postman) #6

GET https://jsonplaceholder.typicode.com/users/1

Send

Test

```
1 javascript
2 pm.test("Status code es 200", function () {
3   pm.response.to.have.status(200);
4 });
```

Body

Headers 24 Test Results

200 OK · 117 ms · 1.4 KB

```
1 {
2   "id": 1,
3   "name": "Leanne Graham",
4   "username": "Bret",
5   "email": "Sincere@april.biz",
6   "address": {
7     "street": "Kulas Light",
8     "suite": "Apt. 566",
9     "city": "Gwenborough",
10    "zipcode": "92998-3874",
11    "geo": {
12      "lat": "-37.3159",
13      "lng": "81.1496"
14    }
15  },
16  "phone": "1-778-736-8031 x56442",
17  "website": "hildegard.org",
18  "company": {
19    "name": "Romaguera-Crona",
20    "catchPhrase": "Multi-layered client-server neural-net",
```

RETO (Validaciones en Postman) #6

GET https://jsonplaceholder.typicode.com/users/1

Send

Query Params

| Key | Value | Description | Bulk Edit |
|-----|-------|-------------|-----------|
| Key | Value | Description | |

Body

Headers 24 Test Results

200 OK · 7 ms · 1.4 KB

Save Response

```
1 {
2   "id": 1,
3   "name": "Leanne Graham",
4   "username": "Bret",
5   "email": "Sincere@april.biz",
6   "address": {
7     "street": "Kulas Light",
8     "suite": "Apt. 566",
9     "city": "Gwenborough",
10    "zipcode": "92998-3874",
11    "geo": {
12      "lat": "-37.3159",
13      "lng": "81.1496"
14    }
15  },
16  "phone": "1-778-736-8031 x56442",
17  "website": "hildegard.org",
18  "company": {
19    "name": "Romaguera-Crona",
20    "catchPhrase": "Multi-layered client-server neural-net",
```

1. ¿Cuál es la diferencia entre GET y POST?

- **GET:** Se usa para **pedir o consultar** información al servidor (como cuando buscas un usuario por su ID). Es como leer un libro; no cambias nada, solo obtienes datos.
- **POST:** Se usa para **enviar o crear** nueva información (como cuando llenas un formulario para registrar un nuevo usuario). Es como escribir una página nueva en el libro.

2. ¿Qué significa el código 200?

Significa "**OK**" o "**Éxito**". Es la respuesta estándar del servidor para decirte que tu petición fue recibida, entendida y procesada correctamente.

3. ¿Qué significa el código 201?

Significa "**Created**" (**Creado**). Es el código que devuelve el servidor específicamente después de una petición **POST** exitosa, confirmando que el nuevo recurso (el nuevo usuario o dato) se creó correctamente.

4. ¿Qué formato tienen las respuestas?

Tienen formato **JSON** (JavaScript Object Notation). Es el estándar de la industria porque es fácil de leer para los humanos (con sus llaves { } y comillas " ") y muy ligero para las computadoras.